RC4 Stream Cipher

1. Introduction

RC4 is one of the most widely known stream ciphers, designed by Ron Rivest in 1987.
It was simple, fast, and efficient, making it popular in software encryption for decades. However, due to vulnerabilities discovered later, RC4 is no longer considered secure for modern use.

RC4 is a **stream cipher** and a **variable-length key algorithm**, which means:

- It encrypts one byte at a time (or larger units like words).
- It generates a **pseudorandom keystream** that is combined with plaintext using the **XOR operation**.

---

2. Working Principle

The main idea of RC4 is to generate a **keystream** (a sequence of random-looking bytes).
Encryption and decryption both use XOR with the keystream:

- **Encryption**:
  `Ciphertext = Plaintext XOR Keystream`
- **Decryption**:
  `Plaintext = Ciphertext XOR Keystream`

Example

```
RC4 Encryption:
Plaintext:    10011000
Keystream:    01010000
Ciphertext:   11001000

RC4 Decryption:
Ciphertext:    11001000
Keystream:     01010000
Plaintext:     10011000
```

As you can see, the same XOR operation is used for both encryption and decryption.

---

3. Structure of RC4

RC4 consists of two main parts:

1. **Key-Scheduling Algorithm (KSA)**
    - Initializes the internal state using the key.
    - Produces a shuffled state array **S** of size 256.
2. **Pseudo-Random Generation Algorithm (PRGA)**
    - Uses the state array to generate a keystream.
    - Continues producing bytes as long as encryption/decryption is required.

---

4. Key-Scheduling Algorithm (KSA)

- RC4 uses a **state vector S** of size 256.
- `S[i]` initially contains values from `0` to `255`.
- Another vector **T** is formed using the key. If the key is shorter than 256 bytes, it is repeated to fill T.

Steps

1. **Initialization**

```
S = list(range(256))
T = [key[i % key_length] for i in range(256)]
```

2. **Permutation of S**

```
j = 0
for i in range(256):
    j = (j + S[i] + T[i]) % 256
    S[i], S[j] = S[j], S[i]
```

After this, **S** is a permuted array based on the key.

---

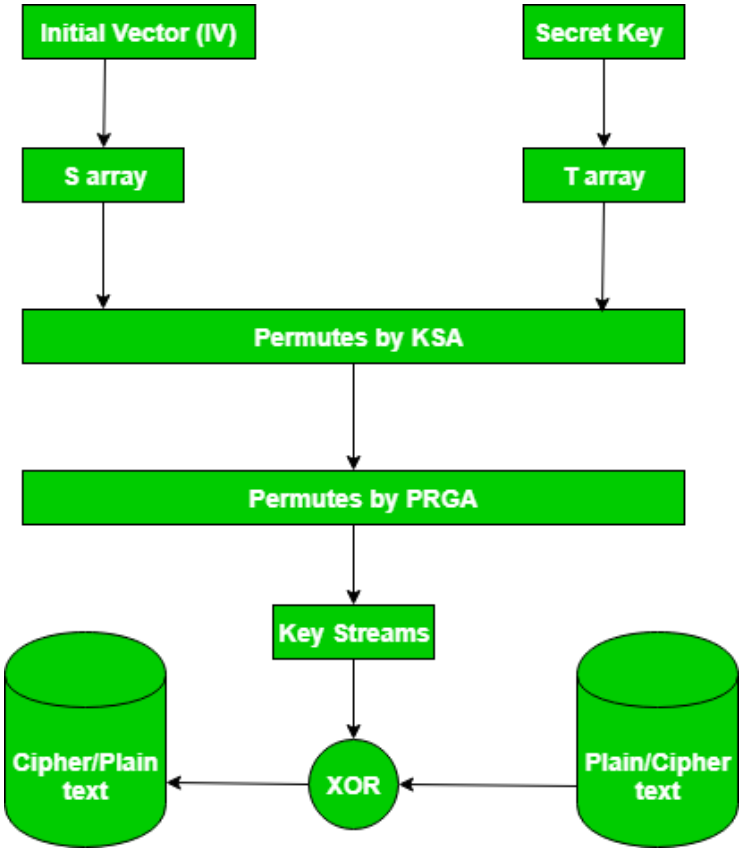
5. Pseudo-Random Generation Algorithm (PRGA)

Once the array S is initialized, the key is no longer used. Instead, keystream bytes are generated continuously.

```
i = j = 0
keystream = []

for _ in range(n):
    i = (i + 1) % 256
    j = (j + S[i]) % 256
    S[i], S[j] = S[j], S[i]
    t = (S[i] + S[j]) % 256
    keystream.append(S[t])
```

- Each generated `k` is XORed with one byte of plaintext to produce ciphertext.
- Process continues byte by byte.



Example

We'll encrypt `"Hello World!"` with RC4.

👉 To keep it simple, I'll use a **short key** (e.g., `"Key"`) so you can follow the steps.

Step 1: Represent Plaintext in ASCII (bytes)

`"Hello World!"` →

```
H   e   l   l   o       W   o   r   l   d   !
72 101 108 108 111 32  87 111 114 108 100 33
```

Step 2: Choose a Key

Let's say our key = `"Key"` → ASCII = `[75, 101, 121]`.

Step 3: Run KSA + PRGA (RC4 engine)

This part generates the **keystream**.
(We won't do all 256 swaps here — that's too long for lecture — instead I'll use the standard RC4 keystream generated for `"Key"`).

👉 For `"Key"`, the first keystream bytes are:

```
byte 1: 0xEB (235)
byte 2: 0x9F (159)
byte 3: 0x77 (119)
byte 4: 0x81 (129)
byte 5: 0xB7 (183)
byte 6: 0x34 (52)
byte 7: 0xCA (202)
byte 8: 0x72 (114)
byte 9: 0xA7 (167)
byte 10: 0x19 (25)
byte 11: 0x0D (13)
byte 12: 0x6F (111)
```

Step 4: XOR Plaintext with Keystream

Now we XOR each plaintext byte with the keystream byte:

Plaintext	ASCII	Keystream	XOR Result	Cipher (Hex)
H	72	235	163	A3
e	101	159	250	FA
l	108	119	27	1B
l	108	129	237	ED
o	111	183	216	D8
(space)	32	52	20	14
W	87	202	149	95
o	111	114	3	03
r	114	167	213	D5
l	108	25	117	75
d	100	13	105	69
!	33	111	78	4E

Step 5: Ciphertext (in Hex)

So `"Hello World!"` under RC4 with key `"Key"` becomes:

```
A3 FA 1B ED D8 14 95 03 D5 75 69 4E
```

Notice: If you XOR this ciphertext with the same keystream, you'll get back `"Hello World!"`.

---

6. Features of RC4

- **Symmetric key**: Same key for encryption and decryption.
- **Stream cipher**: Encrypts byte-by-byte, not in blocks.
- **Variable key size**: From 40 bits to 2048 bits.
- **Speed**: Very fast and efficient.
- **Implementation**: Simple, suitable for both hardware and software.
- **Use cases (historical)**: SSL, WEP (Wi-Fi), VPNs, and file encryption.

7. Advantages

- **Fast and efficient** (suitable for real-time applications).
- **Simple to implement** (few lines of code).
- **Flexible key sizes**.
- **Once very widely used** in network protocols and file encryption.

---

8. Disadvantages & Vulnerabilities

- **Bias in keystream**: The first few bytes of the keystream are not random and can leak information.
- **Key recovery attacks**: Weaknesses make it possible for attackers to recover the key under certain conditions.
- **Security weaknesses**: Less secure than modern ciphers like AES or ChaCha20.
- **Deprecated**: In **2015**, Microsoft officially ended RC4 support in Internet Explorer and Edge.

---

9. Example in Practice

Suppose the keystream generated is:

```
Keystream:  01100101
Plaintext:  10101100
Ciphertext: 11001001
```

For decryption, we just apply XOR again with the same keystream to recover the plaintext.

---

10. Conclusion

- RC4 was once a popular encryption algorithm due to its simplicity and speed.
- It laid the foundation for understanding **stream ciphers**.
- However, due to **serious vulnerabilities**, RC4 is no longer recommended.
- Today, alternatives like **AES in CTR mode** or **ChaCha20** are used instead.

Difference between OTP, Vernam Cipher, and ARC4

Feature	One-Time Pad (OTP)	Vernam Cipher	ARC4 (RC4)
Invented	1917 (used in military)	1917 (Gilbert Vernam)	1987 (Ron Rivest)
Type	Symmetric stream cipher	Symmetric stream cipher	Symmetric stream cipher
Key length	Same length as the message	Usually shorter than the message (repeated)	Variable (40–2048 bits)
Key material	Truly random, used once	Usually not random, reused	Short key expanded into pseudorandom keystream
Encryption method	XOR plaintext with key	XOR plaintext with (repeated) key	XOR plaintext with pseudorandom keystream
Security	**Perfect secrecy** (unbreakable if rules followed)	Weak if key repeats (can be broken with analysis)	Insecure today (biases in keystream, attacks discovered)
Practicality	Impractical (key distribution is hard)	More practical, but not secure	Very practical, very fast (used in SSL, WEP, VPNs, etc.)
Current use	Theoretical / very rare (e.g., spy communications)	Historical interest only	Deprecated (replaced by AES, ChaCha20)

👉 So you can think of it like this:

- **OTP** = *ideal but impractical*.
- **Vernam** = *practical but weak*.
- **ARC4** = *fast software cipher, once popular, now obsolete*.
